FAKULTA
INFORMAČNÍCH
TECHNOLOGIÍ
ČVUT V PRAZE

NI-NLM – Lecture 8

Efficiency: quantization, distillation, low-rank adaptation, MoE.

Zdeněk Kasner

31 Mar 2026

Why efficiency matters

The cost of running LLMs

Modern LLMs are expensive to **train**, **store**, and **run**:

- **Training**: GPT-4 cost an estimated $100M+ in compute.
- **Inference**: serving billions of queries per day requires massive GPU clusters.
- **Storage**: a 70B model in float32 takes $\approx$ 280 GB of disk space.
- **Memory**: loading even a 7B model requires $\approx$ 14 GB of GPU memory.

Question

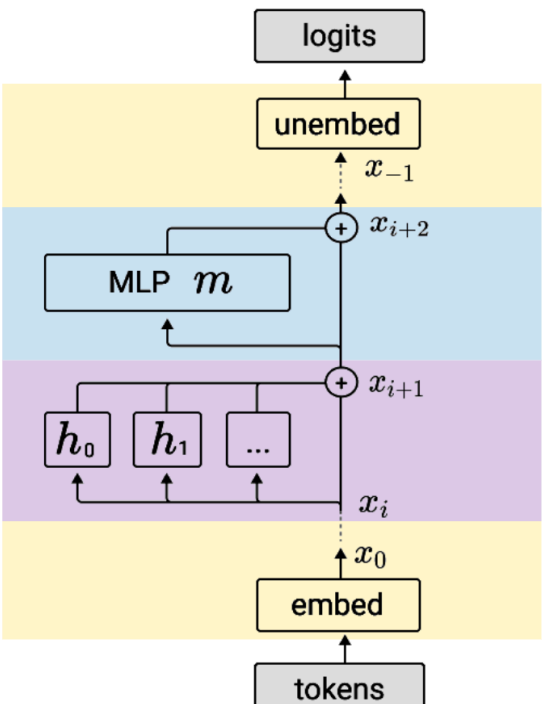
Can you think of ways how to make LLMs smaller, faster, and cheaper (without sacrificing too much performance?)

Transformer: algorithmic complexity

Variables: hidden state dimension (D), sequence length (N), vocabulary size (V).

	Time	Space
final projection (unembedding)	$N \cdot V \cdot D$ each state projected to logits over the vocabulary	$V \cdot D + V \cdot N$ (un)embedding matrix 📦 output logits 🧮
MLP	$N \cdot D^2$ projection matrix 📦	$D^2 + N \cdot D$ activations 🧮
multi-head attention	$N \cdot D^2 + N^2 \cdot D$ queries, keys, values attention matrix 📦	$N^2 + N \cdot D + D^2$ projected values 🧮 Q, K, V matrices 📦
embedding	$N \cdot D$ each token is embedded	$V \cdot D + N \cdot D$ embedding matrix 📦 embeddings 🧮

📦 = model parameters, 🧮 = computed values



source: Transformer circuits

LLM bottlenecks: computational throughput

Time complexity

$$N \cdot D^2 + \boldsymbol{N^2} \cdot \boldsymbol{D} + N \cdot V \cdot D$$

long context → $N^2 \gg D^2$

happens only once

Computing **multi-head attention** becomes the main bottleneck when $N^2 \gg D^2$ (long context).

Solution: efficient attention, mixture of experts.

Typical hyperparameter values:

Param	Min	Max
N	512	128,000+
D	768	4,096+
V	30,000	100,000+
blocks	8	128+

LLM bottlenecks: disk space

Space complexity: model parameters 📦

$$V \cdot D + D^2$$

stored only once

MLPs (→ D^2) typically take up the majority of model parameters – they are repeated in each layer, unlike the embedding matrix.

Solution: parameter quantization, (Q)LoRA.

Typical hyperparameter values:

Param	Min	Max
N	512	128,000+
D	768	4,096+
V	30,000	100,000+
blocks	8	128+

LLM bottlenecks: computational throughput

Space complexity: activations 🧮

$$\boldsymbol{N^2} + N \cdot D + N \cdot V$$

gets more important
with growing input context size

For loading a model to GPU memory, we need to account for both model parameters (fixed) and computed activations (depend on the input context size).

Solution: parameter quantization, (Q)LoRA.

Typical hyperparameter values:

Param	Min	Max
N	512	128,000+
D	768	4,096+
V	30,000	100,000+
blocks	8	128+

LLM bottlenecks: training vs. inference

In terms of resource requirements: **pre-training** $\gg$ **finetuning** $\gg$ **inference**. On the other hand, we typically want to run inference on consumer hardware.

Dataset size

- Pre-training: 10T tokens $\approx$ 50 TB.
- Finetuning: up to 100s GB.

Memory requirements

- **Inference**: 2 bytes per parameter → 14 GB for a 7B model.
- **Training**: 2 bytes per parameter + 2 bytes per gradient + 12 bytes per optimizer state (Adam) → **112 GB** for a 7B model.

Efficient algorithms: overview

Algorithm	What it targets	Train. memory	Train. speed	Inf. memory	Inf. speed
Quantization	Reducing param. size	–	–	✓✓	✓
Distillation	Model compression	–	–	✓✓	✓✓
MoE	Reducing active params.	–	✓✓	–	✓
Linear attention	Faster attention	✓	✓	✓	✓
FlashAttention	HW-optimized attention	✓✓	✓	✓	✓✓
(Q)LoRA	Efficient finetuning	✓✓	✓	–	–

Quantization

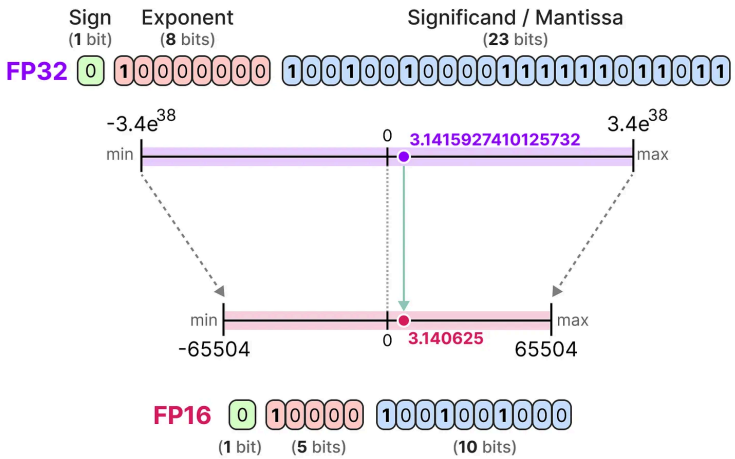
Parameter quantization

source: Grootendorst (2024)

Model parameters are floating point numbers. How do we store them? And can we store them more efficiently?

- **float64** – native Python (typically not used on GPUs).
- **float32** – baseline precision.
- **float16** – half precision.

For inference, float16 typically causes only little performance degradation.



For training, float16 can lead to vanishing gradients → **mixed-precision training**.

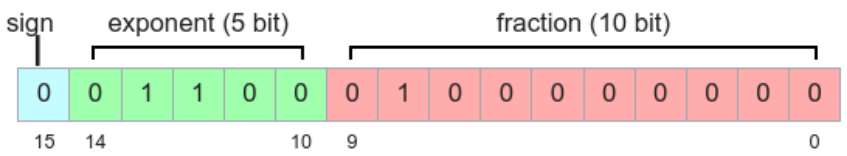
bfloat16

source: Wikipedia

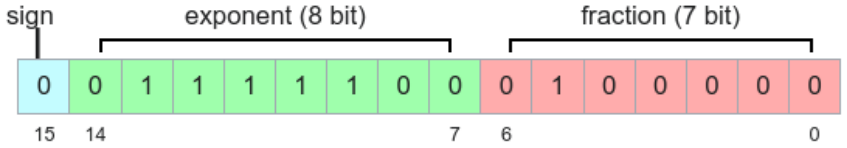
Developed by Google Brain (→ “brain float”).

- A 16-bit float **optimized for ML models**.
- Greater **dynamic range** than float16 (supports outlier weights) at the cost of lower precision.
- Now the **standard format** for training and inference in most LLM frameworks.

IEEE half-precision 16-bit float



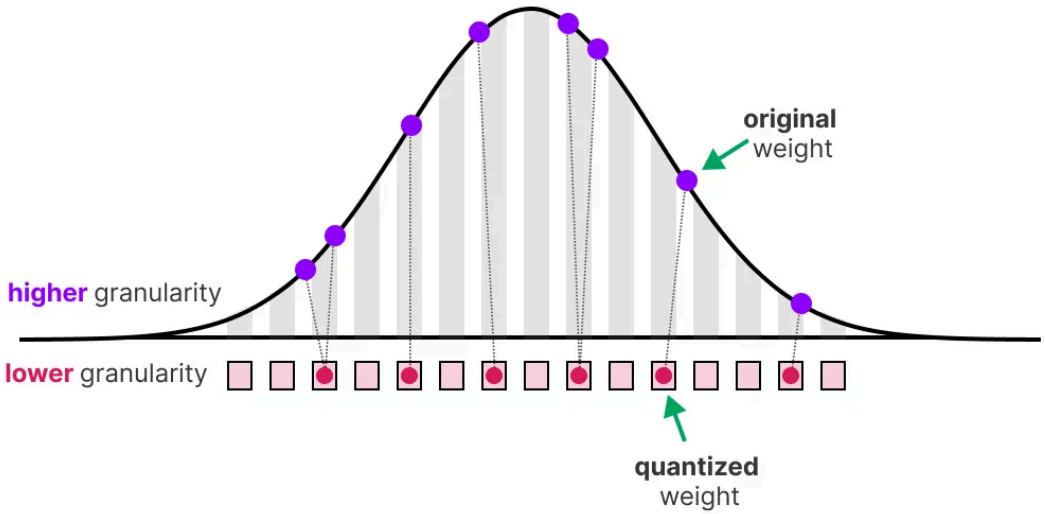
bfloat16



Going further: int8

source: Grootendorst (2024)

Can we go beyond 16-bit? **int8** is an 8-bit integer format – saving twice as much space compared to float16.



But how do we squeeze floating-point weights into the $(-127, 127)$ range?

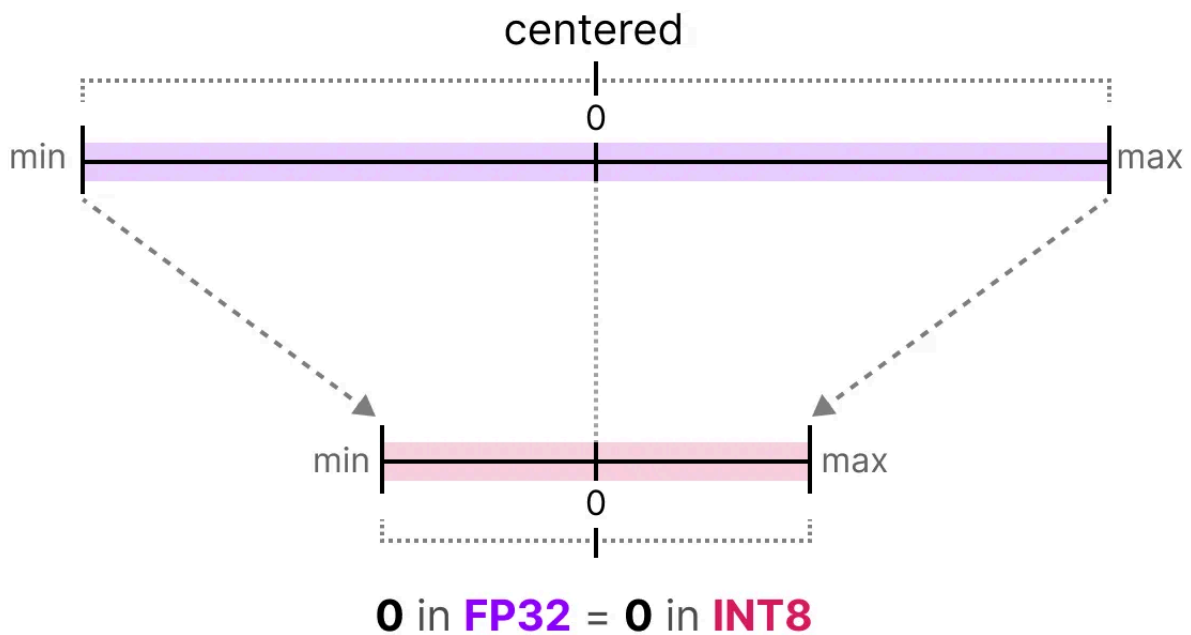
Absmax quantization

source: Grootendorst (2024)

We don't use the full float range – we only use the range of our actual model weights.

We find the **maximum absolute value** and scale all weights proportionally:

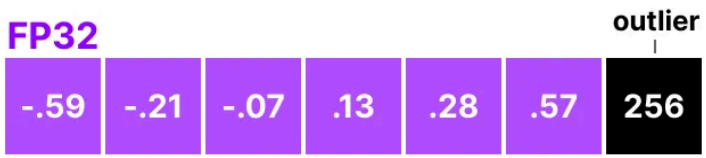
$$x_{\text{quant}} = \text{round}\left(\frac{127}{\max(|\boldsymbol{x}|)} \cdot x\right)$$


centered
0
min
max
min
max
0
0 in FP32 = 0 in INT8

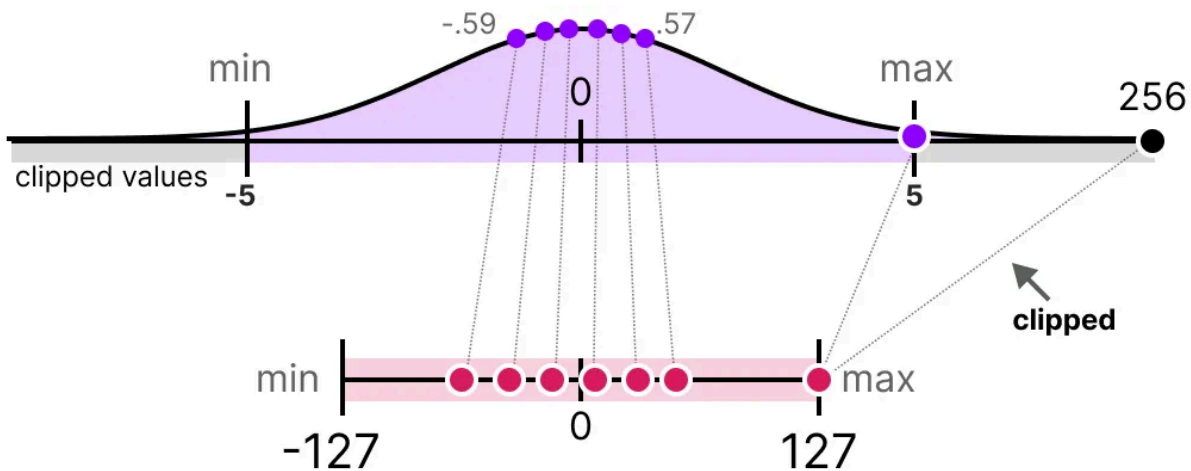
The outlier problem

source: Grootendorst (2024)

What if some of the weights are **outliers**? This would map most of the weights to zero (or near zero):



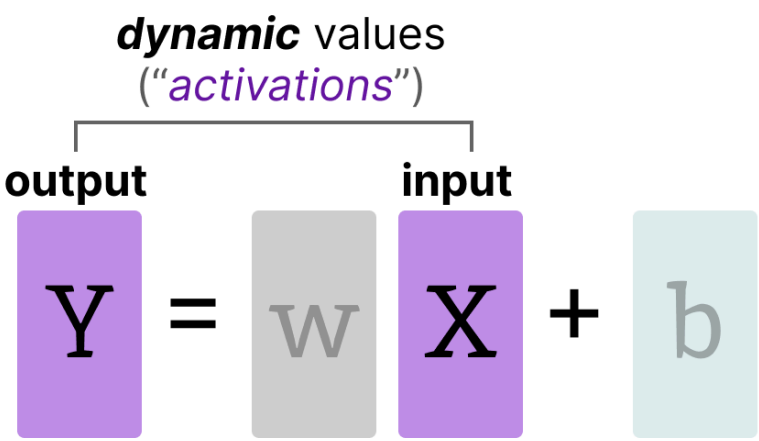
One solution: **clip the outliers**:


-.59
.57
min
0
max
256
clipped values
-5
5
clipped
min
max
-127
0
127

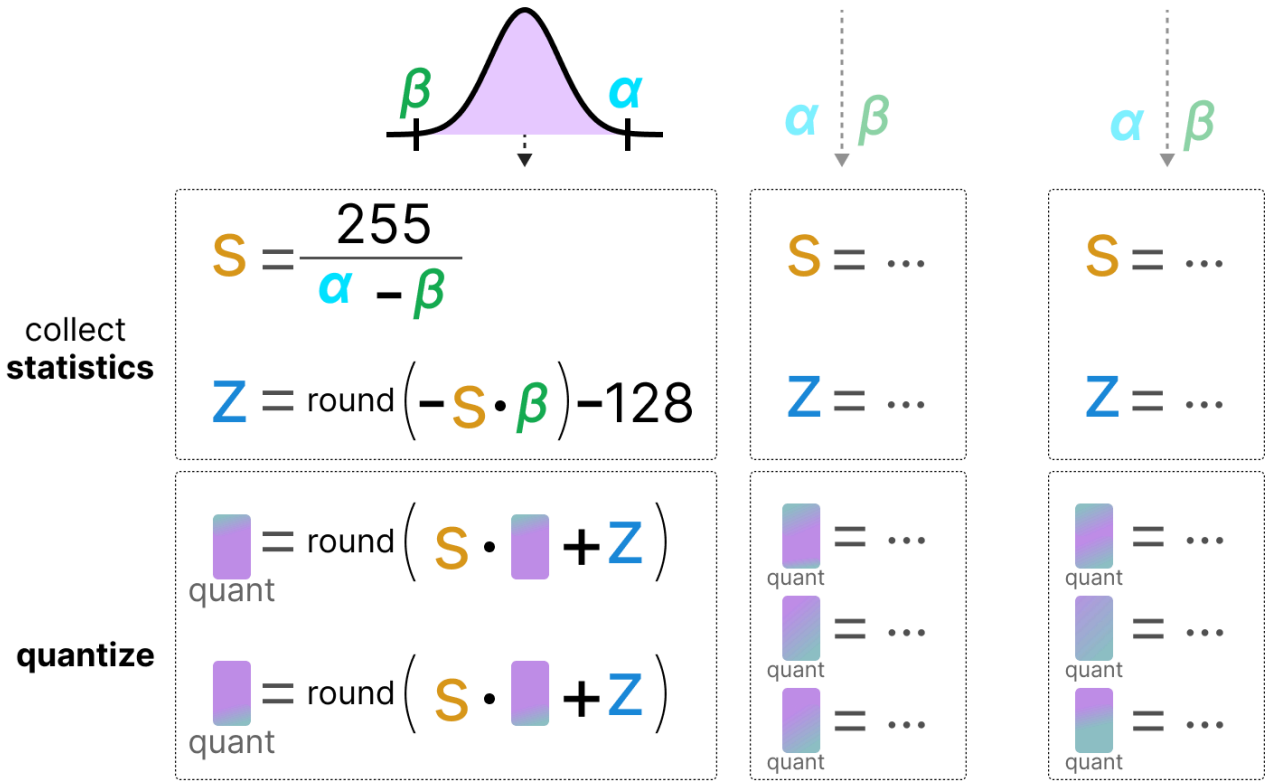
Dynamic quantization for activations

source: Grootendorst (2024)

Model weights can be quantized offline, but **activations are dynamic** – they change with every input.



Solution: compute the quantization range **dynamically** at inference time:


β
α
α
β
α
β
collect
statistics
S = 255 / (α − β)
Z = round(−S · β) − 128
S = ...
Z = ...
S = ...
Z = ...
quantize
quant = round(S · ■ + Z)
quant = round(S · ■ + Z)
quant = ...
quant = ...
quant = ...
quant = ...
quant = ...
quant = ...

Extreme quantization: 1-bit models

source: Ma et al. (2024)

Can we go all the way down to **1-bit**? Each weight is either -1 or $+1$.

BitNet: Scaling 1-bit Transformers for Large Language Models

Hongyu Wang*†‡ Shuming Ma*† Li Dong† Shaohan Huang†
Huaijie Wang§ Lingxiao Ma† Fan Yang† Ruiping Wang‡ Yi Wu§ Furu Wei†◇
† Microsoft Research ‡ University of Chinese Academy of Sciences § Tsinghua University
https://aka.ms/GeneralAI

BitNet (Ma et al., 2024) showed that 1-bit models can achieve competitive performance if trained from scratch with quantization in the loop.

Multiplications become additions → massive speedup on specialized hardware.

Quantization in practice

source: Grootendorst (2024)

GPTQ

- Post-training quantization method.
- Uses extra tricks to go robustly beyond 8-bit (6-bit, 4-bit, 2-bit).
- Calibration on a small dataset.

GGUF

- File format that performs **block-wise quantization**.
- Enables offloading parts of the model to CPU.
- Used by llama.cpp and Ollama.

Note

Models in Ollama are quantized by default to 4-bit.

Quantization: summary

Format	Bits	7B model size	Notes
float32	32	$\approx$ 28 GB	Baseline, rarely used
float16	16	$\approx$ 14 GB	Standard for inference
bfloat16	16	$\approx$ 14 GB	Preferred for training
int8	8	$\approx$ 7 GB	Good accuracy, 2× savings
int4	4	$\approx$ 3.5 GB	Popular for local deployment
1-bit	1	$\approx$ 0.9 GB	Experimental, requires special training

The lower the precision, the more information we lose – but the savings can be dramatic.

⚠️ Rule of thumb

Going from float16 to int4 reduces model size by $\approx 4\times$ with often less than 5% performance drop on standard benchmarks.

Knowledge distillation

What is knowledge distillation?

source: Hinton et al. (2015)

Idea

Train a small **student model** to mimic the behavior of a large **teacher model**, transferring the teacher's "knowledge" into a compact form.

Originally introduced by Hinton et al. (2015) for classification models.

- The student is **smaller and faster** than the teacher.
- The student learns from the teacher's **soft outputs** (probability distributions), not just hard labels.
- These soft outputs carry richer information about the relationships between classes.

Teacher-student framework

source: Hinton et al. (2015)

The student is trained with a combination of two losses:

1. **Distillation loss**: KL divergence between the teacher's and student's output distributions (softened with a temperature T).
2. **Task loss**: standard cross-entropy loss with the ground truth labels.

$$\mathcal{L} = \alpha \cdot \underbrace{D_{\mathrm{KL}}(p_T \parallel p_S)}_{\text{distillation loss}} + (1-\alpha) \cdot \underbrace{\mathcal{L}_{\mathrm{CE}}(y, p_S)}_{\text{task loss}}$$

The **temperature** T controls how soft the distributions are:

- $T = 1$: standard softmax.
- $T > 1$: softer distributions → more information transferred.

Why soft labels matter

source: Hinton et al. (2015)

Consider a classification example:

Hard label

- cat: **1.0**
- dog: 0.0
- car: 0.0

Soft label (teacher)

- cat: **0.7**
- dog: **0.2**
- car: 0.1

The soft label tells the student that "a cat is somewhat similar to a dog, but not to a car" – **the relationships between classes are preserved**.

The same principle applies to LLMs: the teacher's token-level probability distribution carries much more signal than the argmax token alone.

Distillation for LLMs

In the context of LLMs, distillation is used in two main ways:

1. **White-box distillation**: we have access to the teacher's internal representations (logits, hidden states).
   - Train the student to match the teacher's output distribution at each token.
   - Can also match intermediate layer representations.

1. **Black-box distillation**: we only have access to the teacher's generated text.
   - Generate a large dataset using the teacher model.
   - Finetune the student on this synthetic dataset.
   - Much simpler, but less effective.

Note

Black-box distillation is essentially what happens when smaller open models are trained on outputs from GPT-4 or Claude.

Distillation in practice

Distillation is widely used in production:

- **DistilBERT**: 40% smaller, 60% faster, retaining 97% of BERT's performance.
- **Alpaca**: LLaMA 7B finetuned on 52K GPT-3.5-generated instructions.
- **DeepSeek-R1-Distill**: distilled reasoning model available in multiple sizes (1.5B–70B).

⚠️ **Caveat**

TODO: distillation pipeline figure

Many model providers (OpenAI, Google) **prohibit distillation** from their models in their terms of service.

Finetuning & low-rank adaptation

The finetuning problem

We saw that training a 7B model requires $\approx$ **112 GB** of GPU memory (weights + gradients + optimizer states).

This is a lot, even for a single GPU with 80 GB of VRAM (A100).

> **Question**
>
> Can we finetune the model without updating **all** the parameters?

The general idea: **parameter-efficient finetuning (PEFT)** – update only a small number of parameters while keeping most of the model frozen.

Low-rank adaptation (LoRA)

source: Hu et al. (2021)

Hu et al. (2021) proposed **LoRA**: low-rank adaptation of large language models.

Key ideas

- Instead of updating model weights W directly, keep the update ΔW in a **separate matrix**.
- The ΔW matrix has a **low rank** → it can be decomposed into two smaller matrices.

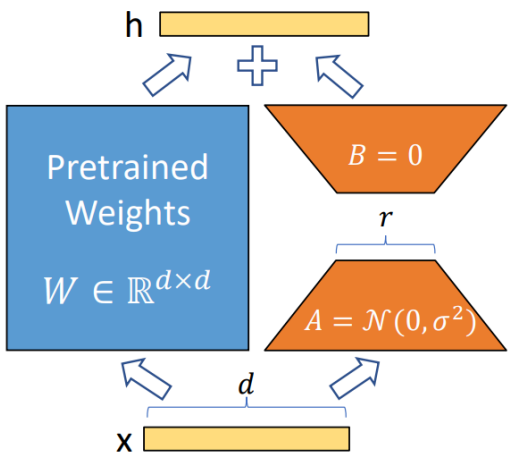
$$W' = W + \Delta W = W + BA$$

where $W \in \mathbb{R}^{d \times k}$, $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, and $r \ll \min(d, k)$.

LoRA: how it works

source: Hu et al. (2021)

- The original model weights W are **frozen** (no gradient updates).
- Only the small matrices A and B are trained.
- For a rank $r = 8$ and $d = 4096$: LoRA adds only $2 \times 4096 \times 8 = 65,536$ parameters per layer instead of $4096^2 \approx 16.7M$.



For inference, LoRA weights can be **merged** into the base model:

$$W' = W + BA$$

→ no additional latency at inference time.

The LoRA weights can be stored as a separate **adapter** (typically a few MB).

QLoRA: finetuning quantized models

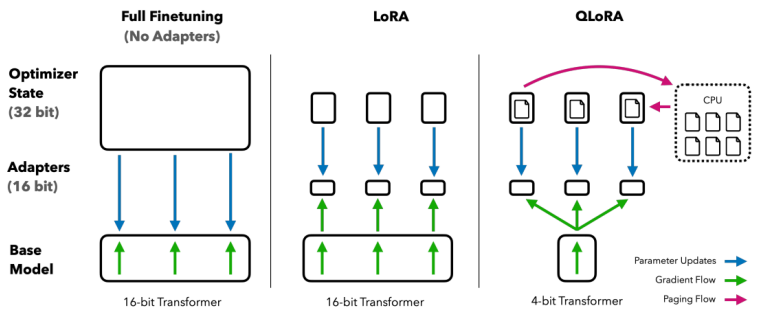
source: Dettmers et al. (2023)

Idea

Can we combine **quantization** and **LoRA** for even higher efficiency?

QLoRA (Dettmers et al., 2023):

- Quantize the base model to 4-bit (**NF4** – NormalFloat4).
- Apply LoRA adapters in higher precision (bfloat16).
- Optimize CPU-GPU memory transfers with **paged optimizers**.



→ Enables finetuning a **65B model on a single 48 GB GPU**.

LoRA and QLoRA in practice

All things being equal: **full finetuning > LoRA > QLoRA** in terms of model performance.

→ The goal is to make the most out of the memory you have.

Method	Bits	Memory (7B model)
Full finetuning	32	≈ 120 GB
Full finetuning	16	≈ 60 GB
LoRA	16	≈ 16 GB
QLoRA	8	≈ 10 GB
QLoRA	4	≈ 6 GB

source: LLaMA-Factory

Frameworks: LLaMA-Factory, HuggingFace PEFT, Unsloth.

Other PEFT methods

LoRA is not the only parameter-efficient finetuning method:

- **Prefix tuning** (Li & Liang, 2021): prepend learnable "virtual tokens" to the input.
- **Prompt tuning** (Lester et al., 2021): similar to prefix tuning, but only for the input layer.
- **Adapters** (Houlsby et al., 2019): insert small bottleneck layers into each Transformer block.
- **DoRA** (Liu et al., 2024): weight-decomposed LoRA – decomposes weight updates into magnitude and direction.

In practice

LoRA and QLoRA remain the **most popular** PEFT methods by far, thanks to their simplicity and strong performance.

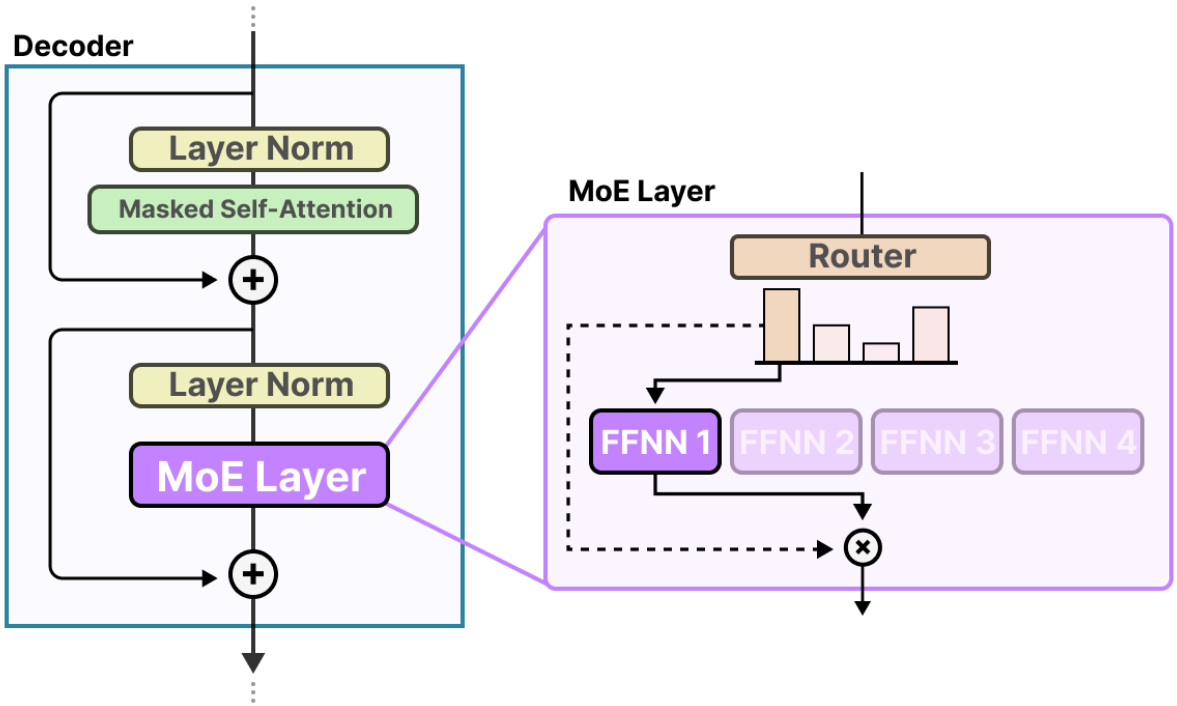
Mixture of experts

Mixture of experts (MoE)

source: Grootendorst (2024)

Idea

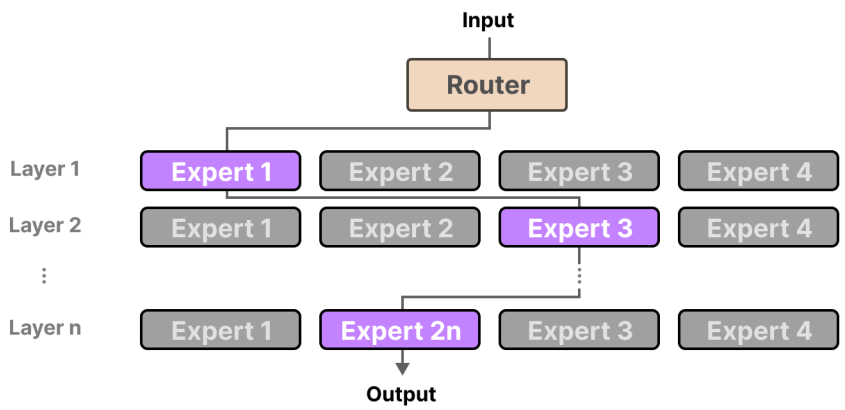
Replace the single feed-forward network (MLP) in each Transformer layer with **multiple experts**. A **router** selects which experts to use for each token.


Decoder
Layer Norm
Masked Self-Attention
Layer Norm
MoE Layer
MoE Layer
Router
FFNN 1
FFNN 2
FFNN 3
FFNN 4

MoE routing

source: Fedus et al. (2024)

We can choose a **different expert at each layer** and a **different set of experts for each token**:



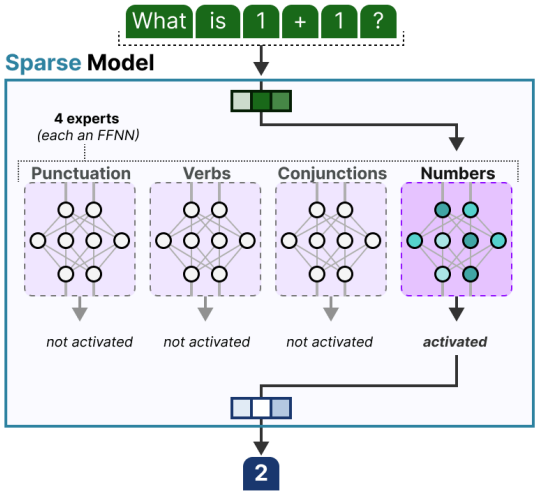

```
class MoeLayer(nn.Module):
    def __init__(self, experts: List[nn.Module],
        super().__init__()
        assert len(experts) > 0
        self.experts = nn.ModuleList(experts)
        self.gate = gate
        self.args = moe_args
```

Why MoE works

source: Grootendorst (2024)

Why is this a good idea?

- Individual experts can **specialize** to solving certain kinds of problems.
- **Faster training** for the same number of total parameters (we only backpropagate through selected experts).
- **Faster inference** (although we still need to load the full model into memory).



Example: Mixtral 8×7B → 8 expert MLPs, approximately equivalent to a **47B dense model**

(not 56B, since attention layers and embeddings are shared).

The router

source: Grootendorst (2024)

The **router** (or "gating network") is a small learned network that decides which experts to activate:

$$G(x) = \mathrm{softmax}\big(\mathrm{TopK}\big(x \cdot W_g\big)\big)$$

- W_g is a learnable weight matrix.
- **TopK**: only the top K experts (typically $K = 1$ or $K = 2$) are activated.
- The router outputs **weights** for combining expert outputs.

⚠️ **Load balancing**

If the router always picks the same experts, some experts are underutilized. MoE models use **auxiliary losses** to encourage balanced expert utilization.

MoE models in practice

MoE has become a very popular architecture for recent LLMs:

Model	Total params	Active params	Experts
Mixtral 8×7B	46.7B	12.9B	8 (top-2)
Mixtral 8×22B	176B	39B	8 (top-2)
DeepSeek-V3	671B	37B	256 (top-8)
Qwen3-235B	235B	22B	128 (top-8)
GPT-4 (rumored)	$\approx$ 1.8T	$\approx$ 220B	16 (top-2)

Key insight

> MoE models can be much larger in **total** parameters while keeping the **active** parameter count (and thus inference cost) comparable to smaller dense models.

MoE: tradeoffs

Advantages

- Faster training and inference for a given quality level.
- Experts can specialize.
- Scales well with more experts.

Disadvantages

- **Memory**: the full model must be loaded (all experts), even though only a few are active.
- **Load balancing**: uneven expert utilization wastes capacity.
- **Communication overhead**: in distributed settings, tokens need to be routed across devices.

Question

MoE models are often paired with **quantization** to address the memory issue.

Why does this make sense?

Efficient attention

Linear attention

source: Schoelkopf (2024)

Standard attention has $O(N^2)$ complexity due to the softmax over all pairs of queries and keys:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(QK^T\right)V$$

Very expensive similarity computation

$$\text{Softmax}\left(QK^T\right)V = \frac{\exp(QK^T)}{\sum_{i=1}^{L} \exp(QK_i^T)} V$$

Standard attention formula

Question

Can we find a cheaper way to compute attention?

Linear attention: rearranging the computation

source: Schoelkopf (2024)

We can replace softmax with a cheaper kernel function φ:

$$\mathrm{Attention}(Q, K, V) = \varphi(Q)(\varphi(K)^\top V)$$

This allows us to **rearrange** matrix multiplications:

Standard: $QK^\top \in \mathbb{R}^{N \times N}$

$\rightarrow O(N^2 \cdot D)$

Linear: $K^\top V \in \mathbb{R}^{D \times D}$

$\rightarrow O(N \cdot D^2)$

The resulting operations are $O(N \cdot D^2)$, which is **linear in sequence length**.

Linear attention: does it work?

source: Schoelkopf (2024)

In theory, linear attention is very attractive (linear vs. quadratic).

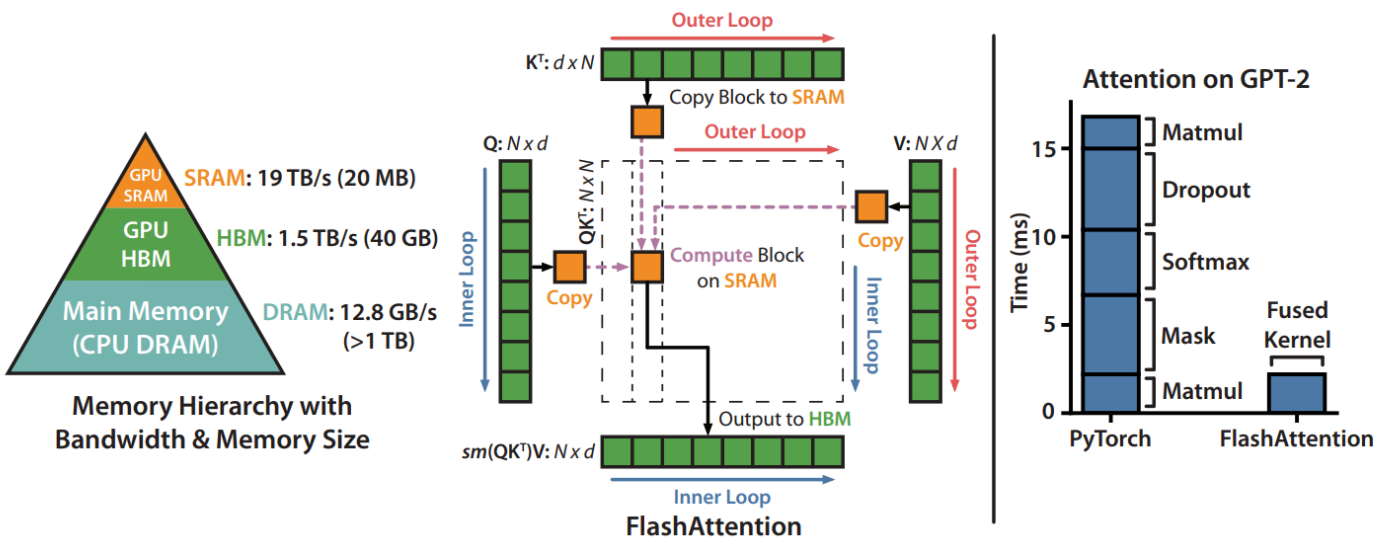
In practice, performance **degrades** compared to full softmax attention.

Recent models like **Mamba** (Gu & Dao, 2023) and **RWKV** use variants of linear attention with additional tricks to close the gap.

FlashAttention

source: Dao et al. (2022)

Approximations like linear attention can degrade performance. Meanwhile, **hardware optimizations** of the full attention mechanism can go a long way.



FlashAttention: how it works

source: Dao et al. (2022)

FlashAttention re-arranges the attention computation to better use the **GPU memory hierarchy**:

- Uses **tiling** to break the attention computation into small blocks that fit in fast GPU SRAM.
- Avoids materializing the full $N \times N$ attention matrix in slow GPU HBM.

Properties:

- **Fast**: 2–3× faster than standard attention.
- **Memory-efficient**: linear in sequence length.
- **Exact**: uses no approximations (unlike linear attention).

- Now **commonly implemented** in all major LLM frameworks (PyTorch, HuggingFace, vLLM, ...).

Summary

Summary

Quantization

- Reduce precision (float16 → int8 → int4).
- Huge memory savings with small accuracy loss.
- GPTQ, GGUF for practical deployment.

Distillation

- Train a small student from a large teacher.

LoRA / QLoRA

- Freeze base model, train low-rank adapters.
- Dramatic reduction in finetuning memory.
- Can be combined with quantization (QLoRA).

- Soft labels transfer more knowledge.
- White-box vs. black-box approaches.

Mixture of experts

- Multiple expert MLPs, only a few active.
- More capacity without proportional compute.
- Used in Mixtral, DeepSeek, Qwen3.

Next lecture

Questions?